



Desafio C++: Sistema de Gerenciamento de Zoológico



Contexto

Você foi contratado por um zoológico local para modernizar o sistema deles. Sua tarefa é criar um programa em C++ orientado a objetos que gerencie os diferentes tipos de animais, respeitando suas características específicas.

Objetivos de Aprendizado

Neste exercício, você deverá aplicar:

1. **Classes e Objetos:** Criação de estruturas básicas.
2. **Encapsulamento:** Proteção de atributos usando `private` / `protected` e métodos `public` (getters/setters).
3. **Herança:** Criação de classes derivadas a partir de uma classe base.
4. **Polimorfismo:** Uso de funções virtuais (`virtual`) e classes abstratas.

Requisitos do Sistema

1. A Classe Base Abstrata `Animal`

Crie uma classe chamada `Animal` que servirá de base para todos os animais do zoológico.

- **Atributos (protegidos ou privados):**
 - `nome` (string)
 - `idade` (int)
- **Métodos (públicos):**
 - Construtor que recebe `nome` e `idade`.
 - Getters e Setters para os atributos.
 - `virtual void emitirSom() const = 0;` (Uma função virtual pura, tornando `Animal` uma classe abstrata).
 - `virtual void exibirInfo() const;` (Imprime o nome e a idade do animal).

2. Classes Derivadas Intermediárias (Opcional, mas recomendado)

Crie classes que herdam de `Animal`:

- **Classe `Mamifero`:**
 - **Atributo extra:** `corDoPelo` (string).
 - Sobrescreva o método `exibirInfo()` para incluir a cor do pelo (lembre-se de chamar o `exibirInfo` da classe base!).
- **Classe `Ave`:**
 - **Atributo extra:** `envergaduraAsa` (float - em metros).
 - Sobrescreva o método `exibirInfo()`.

3. Classes de Animais Específicos

Crie classes concretas que herdam das classes acima (ou diretamente de `Animal`):

- **Classe** `Leao` (Herda de `Mamifero`):
 - Implemente o método `emitirSom()` para imprimir algo como *"O leão ruge: ROAAAR!"*.
- **Classe** `Arara` (Herda de `Ave`):
 - Implemente o método `emitirSom()` para imprimir algo como *"A arara grita: PRRRAAK!"*.
- **Classe** `Cobra` (Herda diretamente de `Animal` ou crie uma classe `Reptil`):
 - **Atributo extra:** `venenosa` (`bool`).
 - Implemente o método `emitirSom()` (*"A cobra sibila: Sssss..."*) e sobrescreva `exibirInfo()`.

4. A Classe `Zoologico` (O Grande Teste do Polimorfismo)

Crie uma classe `Zoologico` para gerenciar as criaturas.

- **Atributos:**
 - Uma lista/vetor de ponteiros para `Animal`: `std::vector<Animal*> jaulas;`
- **Métodos:**
 - `void adicionarAnimal(Animal* novoAnimal);` (Adiciona ao vetor).
 - `void listarAnimais() const;` (Percorre o vetor chamando `exibirInfo()` de cada animal).
 - `void horaDoShow() const;` (Percorre o vetor chamando `emitirSom()` de cada animal).

O Programa Principal (`main.cpp`)

Na sua função `main`, você deve:

1. Instanciar um objeto `Zoologico`.
2. Criar pelo menos um `Leao`, uma `Arara` e uma `Cobra` dinamicamente (usando `new`).
3. Adicionar esses animais ao zoológico.
4. Chamar `listarAnimais()` e `horaDoShow()` do zoológico.
5. **IMPORTANTE:** Lembre-se de liberar a memória alocada (usando `delete`) no final do programa para evitar vazamento de memória (Memory Leak), possivelmente no destrutor da classe `Zoologico`.



Desafio Bônus (Nível Mestre)

Em vez de usar `std::vector<Animal*>`, tente usar ponteiros inteligentes (`std::vector<std::unique_ptr<Animal>>`) introduzidos no C++11. Isso elimina a necessidade de usar `delete` manualmente, gerenciando a memória de forma muito mais segura e moderna!